Stacked Ensemble Modeling for U.S. County-Level Election Outcome Prediction

Zixuan Liu, Xinyang Wang, Yichen Wang, Jessica Zhao

August, 2024

1 Introduction

The 2020 U.S. Presidential Election was marked by heightened political engagement and significant voter turnout amidst a global pandemic and social unrest. This project aims to predict the winner of each county in the 2020 US Presidential Election—either Biden or Trump—using demographic and educational predictors derived from US Census Bureau estimates. Based on the background information and data analysis, we notice the crucial predictor variables include racial composition, age distribution, and educational attainment, all of which are hypothesized to correlate with voting preferences. We will elaborate on our parameter selection process in detail and provide the final model chosen in the following paragraphs. Hopefully, it can offer insights into voter behavior across the United States.

2 Exploratory Data Analysis

In this section, we provide visual graphics that capture essential aspects of the dataset, including analysis of demographics, multicollinearity among predictors, and how potential transformations can be applied. Most importantly, we explore potential relationships between some predictors and the response variable as guidance for conducting further analysis and uncovering more patterns.

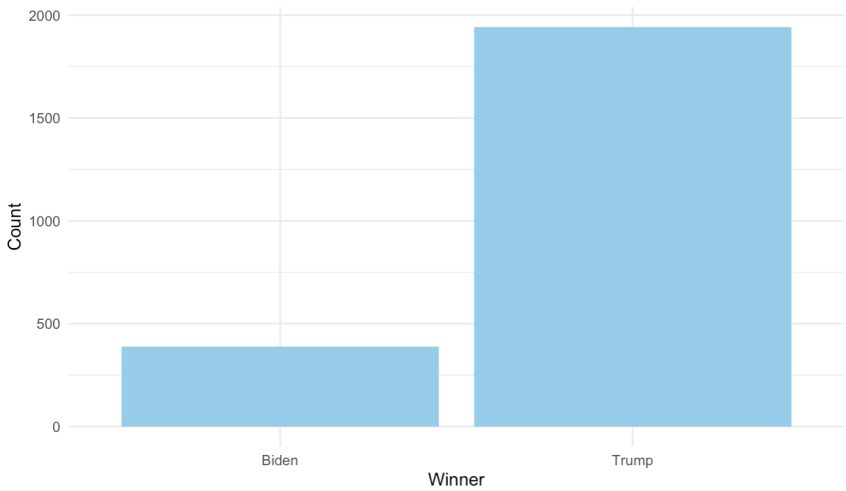

Figure 1: Distribution of Election Winner

Figure 1 represents the distribution of counties won by each candidate in the dataset, identified as either "Biden" or "Trump." The x-axis lists the candidates, while the y-axis indicates the count of counties. The chart shows that Trump has a substantially higher count, with nearly 2000 counties, compared to Biden, who has fewer than 500 counties. This significant difference reflects the number of counties where each candidate won the majority of votes in the election data provided. This visualization highlights the substantial lead in the count of counties won by Trump.

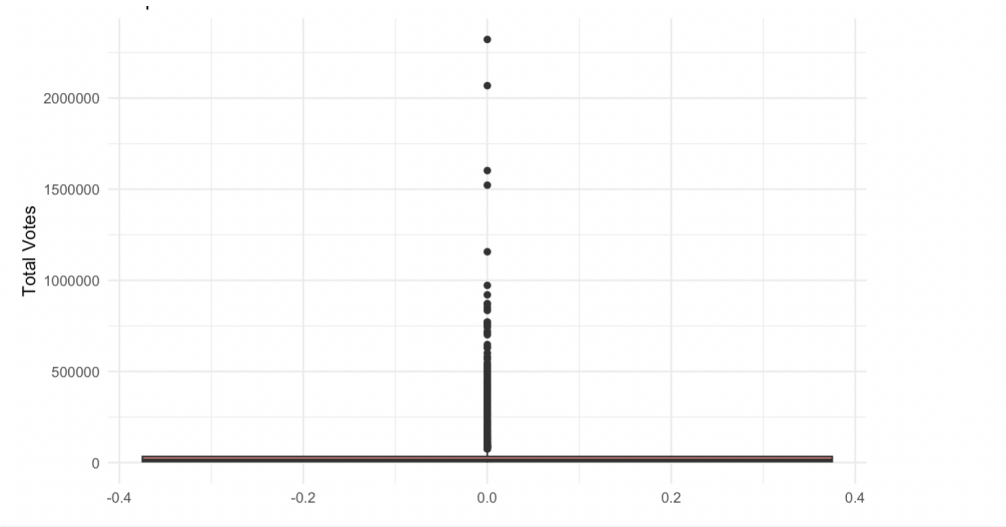

Figure 2: Boxplot of Total Votes

In Figure 2, the box plot visualizes the distribution of total votes across different counties. The y-axis represents the total votes, with a range extending beyond 2,000,000 votes. The plot shows a large number of counties with relatively low total votes clustered near the lower end, with a few outliers stretching significantly higher. These outliers indicate counties with exceptionally high voter turnout. The data distribution suggests a highly skewed pattern, where most counties have low total votes, but a few have much higher totals. It corresponds to the real-world situation accurately.

In Figure 3, the histogram illustrates the distribution of total population (x0001E) across different regions. The x-axis represents the total population, while the y-axis shows the frequency of occurrences. The histogram is filled with a "steel blue" color and has black borders for each bin. The plot reveals a highly skewed distribution, with the majority of regions having a relatively low total population. A significant number of regions fall under the lower end of the population scale, with frequencies rapidly declining as the population increases. This skewness suggests that most regions have smaller populations, with only a few regions having very large populations.

The scatter plot on the left shows a positive linear relationship, with data points trending upward as the female population increases. The range extends from zero to over two million for both variables, indicating that counties with larger female populations generally have higher total vote counts. Similarly, the scatter plot for the male population exhibits a positive linear trend. As the male population increases, total votes also rise, reaching upwards of two million. The distribution is similar to that of the female population plot, with data points clustering near the lower population end but extending linearly with increasing population size.

Notably, there is no significant divergence in patterns between the female and male populations, reflecting a proportional relationship with voter turnout across both groups. This analysis indicates that gender-specific population size does not significantly influence the total votes, as both exhibit

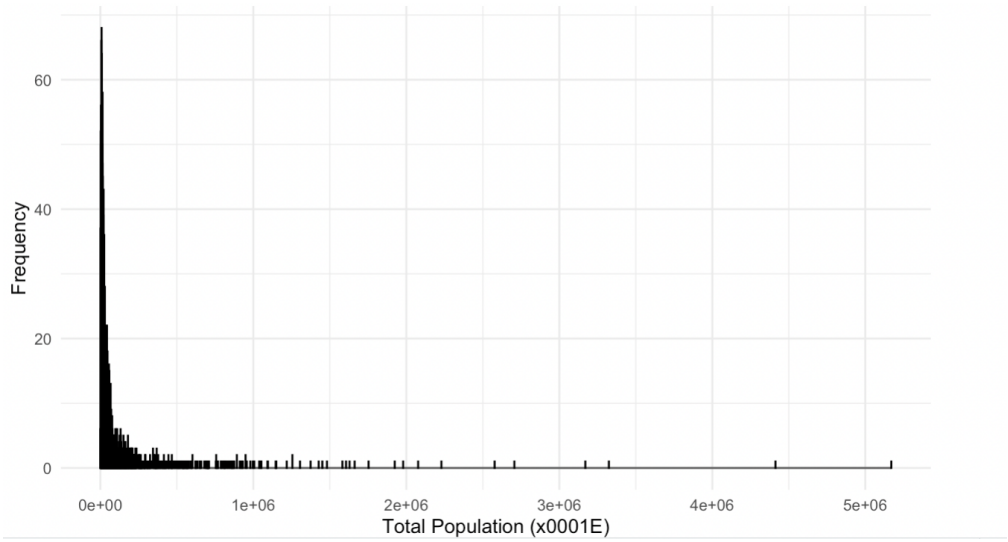

Figure 3: Total Population Distribution

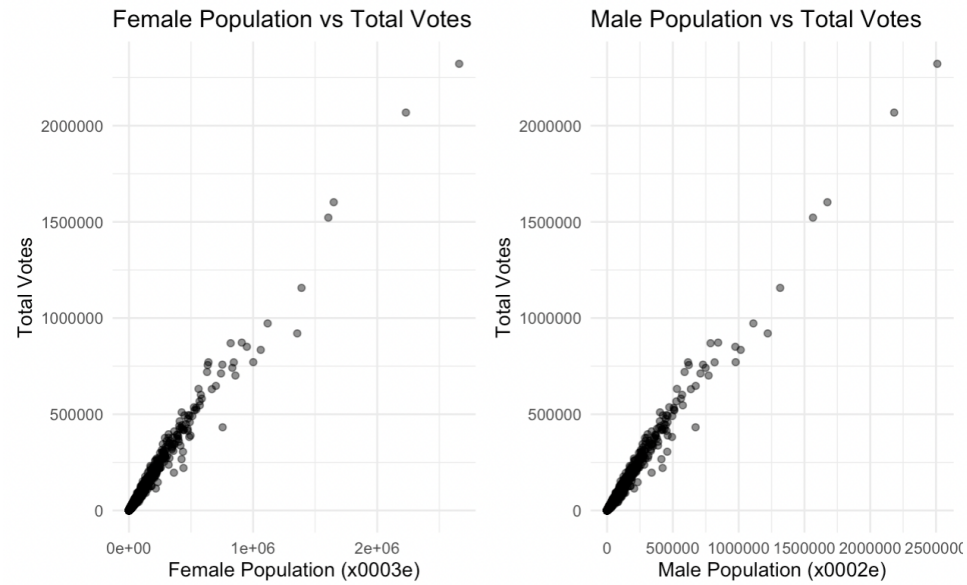

Figure 4: Female/Male Population vs. Total Votes

comparable trends and distributions.

In Figure 5, the scatter plot displays the relationship between income per capita in 2017 and total votes. The x-axis represents income per capita, and the y-axis shows total votes. The data points, shown in dark green, indicate a clustering of counties at lower income levels with a modest positive correlation between income and total votes. The plot reveals a slight increase in total votes with higher income per capita, but this trend plateaus beyond 100,000 dollars, suggesting a limited influence of high income on total votes.

The scatter plot in Figure 6 illustrates the relationship between the percentage of the White population and total votes. The x-axis represents the percentage of the White population, while the y-axis shows the total votes. The plot includes a red linear regression line, indicating a strong

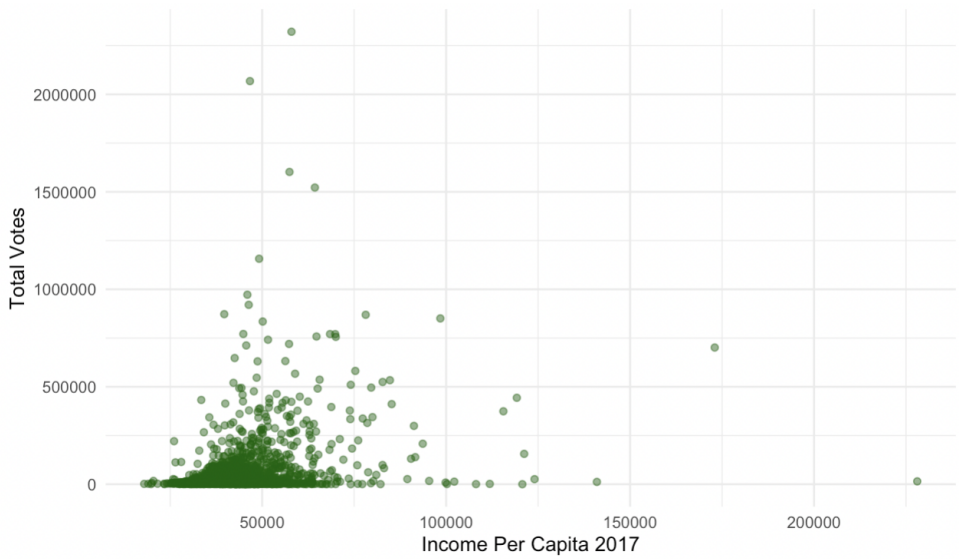

Figure 5: Income Per Capita 2017 vs Total Votes

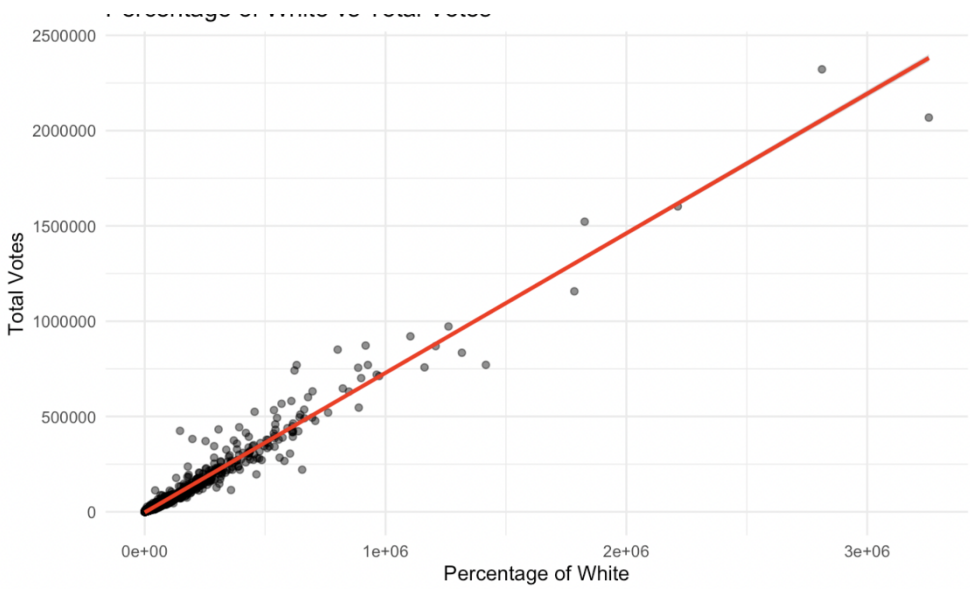

Figure 6: Percentage of White vs Total Votes

positive correlation. As the percentage of the White population increases, the total votes also rise significantly, suggesting a strong association between these two variables.

In Figure 7, the scatter plot depicts the relationship between the percentage of the Black or African American population and total votes. The x-axis represents the percentage of the Black or African American population, and the y-axis shows total votes. Similar to Figure 6, a red linear regression line highlights a positive correlation. However, the data points show more variability, and the distribution is less concentrated than in the White population plot. This indicates a positive, albeit weaker, relationship compared to the White population, with total votes increasing as the percentage of the Black or African American population rises.

The pair plot in Figure 8 visualizes the relationships between the first five demographic predictors:

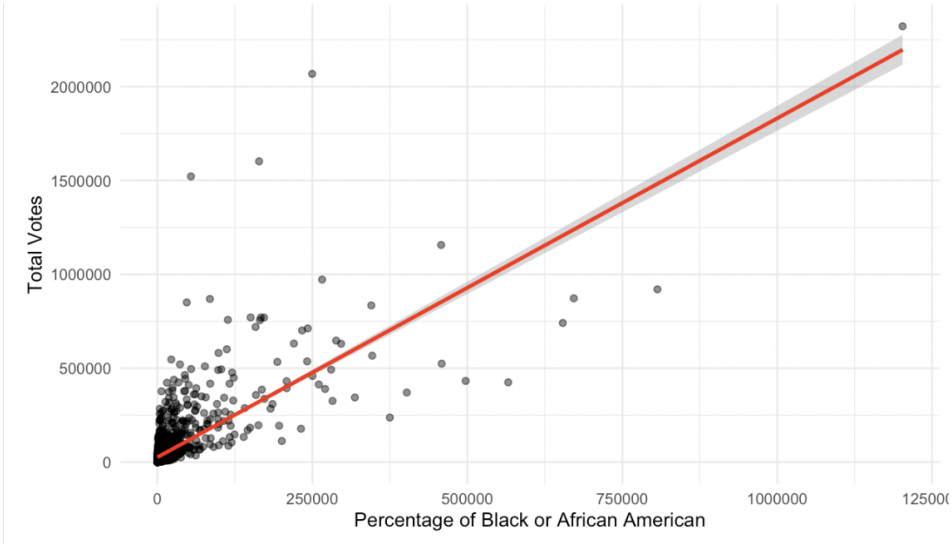

Figure 7: Percentage of Black or African American

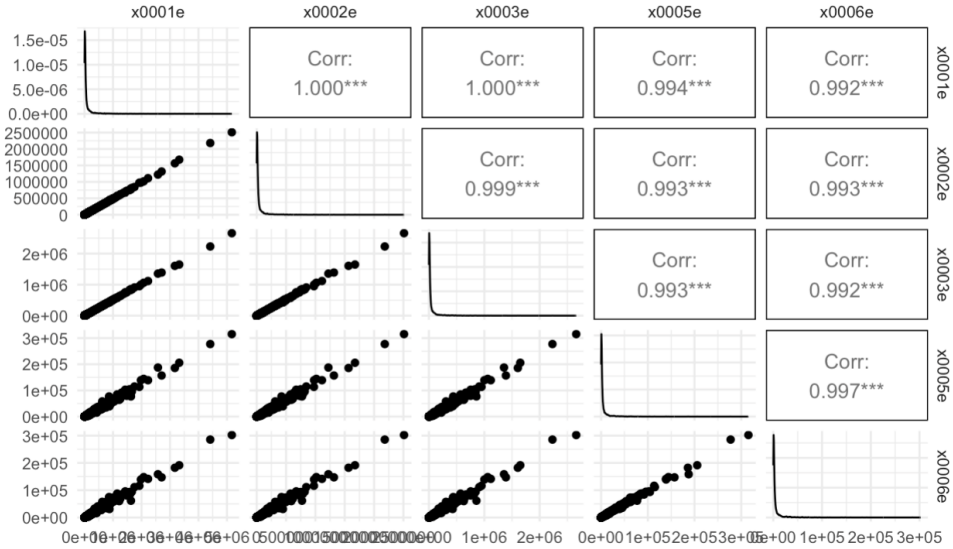

Figure 8: Multicollinearity Plot for First 5 Predictors

total population (x0001E), male population (x0002E), female population (x0003E), the population under 5 years (x0005E), and the population from 5 to 9 years old (x0006e). The matrix includes scatter plots and correlation coefficients for each pair of variables. The high correlation values, all above 0.993, indicate extremely strong multicollinearity among these predictors. We must be cautious about them during modeling since it could potentially worsen the model's effectiveness and lead to overfitting when building regression models.

In Figure 9, the histogram represents the distribution of the log-transformed total votes. The x-axis denotes the log-transformed total votes, while the y-axis shows the frequency of occurrences. The histogram bars are filled with "lightblue" color and outlined in black. The log transformation was applied to address the skewness in the original total votes data. It seems very effective since the distribution appears more symmetric and approximately bell-shaped, centering around a log value

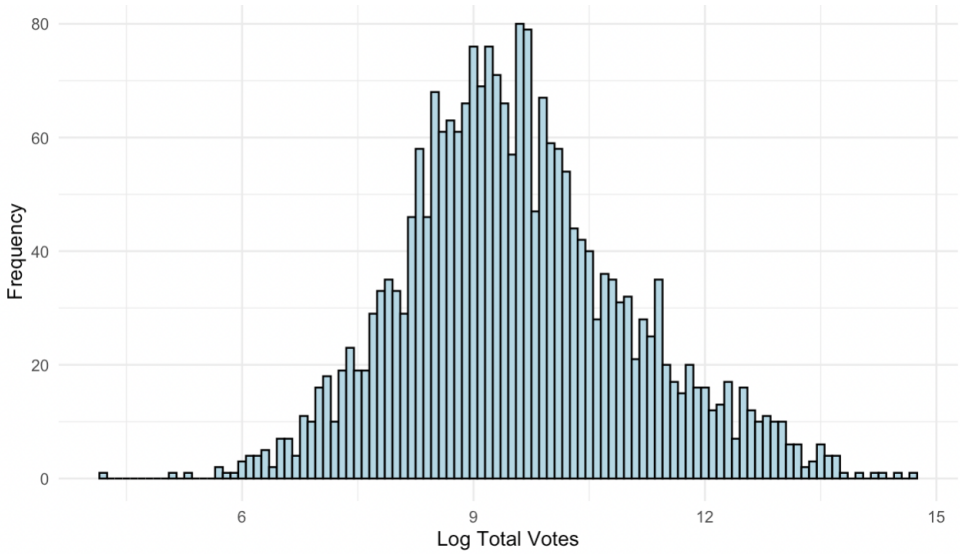

Figure 9: Log-Transformed Total Votes Distribution

of 10.

3 Data Preprocessing

Through examining the whole training dataset, we find there are missing entries in columns `income_per_cap_2016`, `income_per_cap_2017`, `income_per_cap_2018`, `income_per_cap_2019`, `income_per_cap_2020`, `gdp_2016`, `gdp_2017`, `gdp_2018`, `gdp_2019`, `gdp_2020`. To deal with this, we utilized mutate() function to implement mean imputation for all NA values in the training set. Next, we set the seed for reproduction and split the data into 10 groups of roughly equal size using `vfold_cv` function, where strata is winner to be predicted. Then we created the recipe containing steps that removes the zero-variance variables, converts categorical variables into dummy variables, and normalizes numerical variables. In this manner, we ensure there are no missing values in the training set and all variables are properly preprocessed for various classification models.

4 Candidate models and model tuning

4.1 Candidate Models

Through analyzing the features of the training and testing dataset, we decide to implement six candidate models: Logistic Regression, Random Forest, XGBoost, K-Nearest Means(KNN), Support Vector Machine(SVM), and stacking (XGBoost, random forest, and logistic). As these six models are popular and have been widely adopted in various classification problems, we learned their pros and cons, wrote out their basic versions, and measured their performance. Detailed descriptions of each model are as follows:

Logistic Regression: Logistic Regression predicts categorical outcomes with a logistic function. It is relatively simple and has high efficiency in scenarios where the outcome is binary dependent on the predictors. After initial testing, we found that this model perform generally well in predicting

the testing data. Through data tuning, we optimized penalty and mixture parameters to enhance the model's accuracy on modeling the training dataset. The optimal model uses a penalty parameter of 0.0000248 with a model accuracy of 0.91213.

Random Forest: The Random Forest model operates by constructing multiple decision trees during training and outputs the mean prediction of the individual trees. It has the advantages of accurate prediction and robustness to overfitting. When implementing the model, we first constructed the model with hyperparameters mtry, min_n, and trees along with the workflow and the pre-constructed recipe. Specifically, we preset trees to be 2000, then let the tune_grid() function determine the other two. Unfortunately, the random forest has been outperformed by many other candidate models, and the accuracy score is merely 0.878.

XGBoost: XGBoost is a powerful and efficient implementation of gradient boosting, designed specifically for speed and performance. It is highly effective in classification tasks due to its ability to handle large datasets and accommodate complex nonlinear relationships. We configured the XGBoost model with several hyperparameters including mtry, trees, min_n, tree_depth, learn_rate, loss_reduction, and sample_size. Each parameter was carefully adjusted to refine the model's ability to predict accurately while avoiding overfitting. Through fine-tuning and testing, XGboost gives out a test accuracy of 0.890, which is still lower than that of our final stacked model.

K-Nearest Means: The K-Nearest Neighbors algorithm is a non-parametric method employed for both classification and regression tasks, using the closest k training examples in the feature space to predict the output. It is known for capturing complex patterns and its simple and intuitive working algorithm. When implementing the KNN model, we created the model with neightbors, weight_func, and dist_power to be hyperparameters, and we added a series of preprocessing steps in the recipe. Through a combination of 'grid_random' and 'tune_grid' for hyperparameter tuning, we identified optimal settings: neighbors = 15, weight_func = "rectangular", and dist_power = 1. Though it generally perform well, KNN does not outperform XGBoost and Logistic Regression, achieving only 87% accuracy in predicting the testing dataset.

Support Vector Machine: The Support Vector Machine model, particularly with its Radial Basis Function (RBF) kernel, is a robust classifier renowned for its effectiveness in high-dimensional spaces. Utilizing the `kernlab` engine and operating in classification mode, our implementation of SVM was designed to dynamically adjust the 'cost' and 'rbf_sigma' parameters to optimize decision boundaries. With parallel computing, we utilize a random grid search across specified ranges for 'cost' and 'rbf_sigma'. The tuning, managed on a cluster reducing the core count by one to avoid overloading the host machine, involved iterating through 30 unique combinations of hyperparameters. After rigorous evaluation using 'tune_grid' with metrics focused on ROC AUC and accuracy, the best parameters were selected based on ROC AUC performance. The results of this model does not outperform XGBoost and Logistic Regression, so we would not consider to use it as the final model.

Stacking (XGBoost, random forest, and logistic): Stacking is an advanced ensemble technique that combines multiple classification models to improve predictive performance, leveraging the strengths of each individual model. In our implementation, we utilized XGBoost, Random Forest, and Logistic Regression as base learners, each configured with a unique set of hyperparameters optimized for classification tasks. The stacking process began with preprocessing the training data, including mean imputation for missing values and normalization of numeric predictors. We created

Table 1: Performance Comparison

Model	**Type of model**	**Engine**	**Recipe**	**Hyperparameters**
Model 1	Logistic Regression	ranger	Convert to dummy, normalize numeric, impute mean for missing	penalty = 0.0000248, mixture = 0.775
Model 2	Random Forest	lm	Convert to dummy, normalize numeric, impute mean for missing	trees = 2000
Model 3	XGBoost	xgboost	Convert to dummy, normalize numeric, impute mean for missing	mtry = 40, trees = 100, tree depth = 10, learn rate = 0.1
Model 4	KNN	kknn	Convert to dummy, normalize numeric, impute mean for missing	neighbor = 15, weight func = "rectangular", dist power = 1
Model 5	Support Vector Machine	kernlab	Convert to dummy, normalize numeric, impute mean for missing	cost = 3.33, rbf sigma = 1.11
Model 6	Stacking(XGBoost, random forest, logistic)	glmnet, ranger, xgboost	Convert to dummy, normalize numeric, impute mean for missing	mtry = 17,min n = 50,trees = 230, tree depth = 6,loss reduction = 1.176322,learn rate = 0.1,sample size = 0.9968396, stop iter = 5

separate workflows for each model: a tuned boost tree model using `xgboost`, a tuned random forest model using `ranger`, and a logistic regression model using `glm`. After tuning the hyperparameters of the tree-based models through grid search and fitting the logistic model, all models were integrated into a single stack using the `stacks` package. Later on, we further manually fine-tuned the hyperparameters based on given tuned results and thus discarded the tuning phase. This setup allowed us to blend predictions and fit member models to the stack, ultimately refining the combined model's accuracy. The stacked model was finalized and utilized for class prediction.

4.2 Model evaluation and tuning

For cross-validation, We use 10-fold to measure the performance of each model, since v=10 is a robust setting to evaluate the performance and stability of predictive models.

For **Logistic Regression,** We used a grid search to tune two hyperparameters of the logistic regression model: `penalty` and `mixture`. The `penalty` parameter controls the regularization strength, while the `mixture` parameter determines the balance between L1 (lasso) and L2 (ridge) regularization. We utilize `tune_grid` function performs the grid search with two primary metrics to evaluate: ROC AUC and accuracy. ROC AUC assesses the model's ability to distinguish between classes, while accuracy measures the proportion of correctly classified instances. The best hyperparameters were selected based on the highest ROC AUC score from the cross-validation results. The optimal values were found to be a penalty of 0.0000248 and a mixture of 0.775. These parameters were used to finalize the logistic regression model, ensuring optimal performance. The accuracy achieved by the logistic model is pretty high with 0.9039112 in prediction. This is considered to

be a strong candidate model . However, the only two hyperparameters make the model hard to be tuned to perform more advanced potentially.

For **KNN**, We optimized the model by tuning three key hyperparameters: `neighbors` (number of nearest neighbors), `weight_func` (weighting function for neighbors), and `dist_power` (power parameter for distance calculation). A random grid search was employed to explore a range of values: `neighbors` from 1 to 50, `weight_func` with "rectangular," "triangular," and "epanechnikov," and `dist_power` from 1 to 2. Parallel processing was utilized to expedite the grid search. The primary evaluation metric was ROC AUC. Besides the tune we also do the fine tuning to enhance the performance. We utilize the combination of neighbors = 15, weight_func = "rectangular", and dist_power = 1 and find that the overall performance is in robust 87% . Considering the computation expense of the KNN model in tuning, KNN model is not a generally ideal model compared to other models.

For our **XGBoost Model**, we adjusted several hyperparameters including the number of trees, the maximum features considered at each split (`mtry`), the minimum sum of instance weight needed in a child (`min_n`), the maximum depth of a tree (`tree_depth`), the learning rate (`learn_rate`), the minimum loss reduction required to make a further partition on a leaf node (`loss_reduction`), and the fraction of the training data sampled for building trees (`sample_size`). Each of these parameters plays a critical role in the model's ability to learn from the training data effectively. The number of trees and `mtry` influence the model's complexity and ability to generalize, while the depth and learning rate control the learning process's granularity and speed. Through a grid search involving various combinations of these hyperparameters, we fine-tuned the model to achieve optimal performance. The final settings were determined to be 100 trees, `mtry` of 40, `min_n` of 10, `tree_depth` of 10, `learn_rate` of 0.1, `loss_reduction` of 0.01, and a `sample_size` of 1. During the tuning process, we utilized parallel processing with the `doParallel` package to efficiently manage computational resources, allocating all but one core of a computing cluster to prevent system overload.

As for the **Random Forest model**, the initial tuning involved a grid search exploring various settings for `mtry`, `min_n`, and `trees`. `mtry` gives the number of features considered for splitting at each node, influencing model diversity and prediction accuracy. `min_n` refers to the minimum number of samples required to split a node, affecting the tree's depth and complexity. The number of `trees` specifies the number of trees in the forest, where more trees generally improve model performance but also increase computational cost. We set tune() for mtry and min_n, and set tree to be 2000. The tuning results in a csv with accuracy of around 0.88, far lower than the result produced by other models.

As for the **SVM**(support vector machines), we constructed the svm_rbf model from parsnip, setting hyperparameters cost and rbf_sigma to be tune(). The 10-fold validation set are conducted in the same way as done in all other models. In the grid_random function, we set the tuning range for both hyperparameters to be (0.001, 10) and (0.001, 1), and we got 3.327245 and 1.109683 respectively for cost and rbf_sigma after running the code. However, the ultimate accuracy score for SVM is conspicuously lower than other models, which is merely 0.81146, making it uncompetitive in our model selection.

The ultimate model we choose, which is the **Stacked model consisted of a Boosted Tree, a Random Forest, and a Logistic Regression** in order, is a complex and more robust model

utilizing the stacking technique of tidymodel. We first set tuning for all hyperparameters including the mtry, min_n, trees, tree_depth, loss_reduction, learn_rate, and sample_size in XGboost model as well as mtry, min_n, trees in Random Forest model. Based on the preliminary tuning results, we did a series of manual fine-tune aiming at attaining a higher accuracy score for the test set. Eventually, through our consecutive explorations and experiments, the hyperparameter combination of mtry = 17, min_n = 50, trees = 230, tree_depth = 6, loss_reduction = 1.176322, learn_rate = 0.1, sample_size = 0.9968396, stop_iter = 5 for XGboost, and mtry = 11, min_n = 50, trees = 120 for Random Forest achieves the optimal accuracy score 0.92236, which is substantially higher than that of other candidate models.

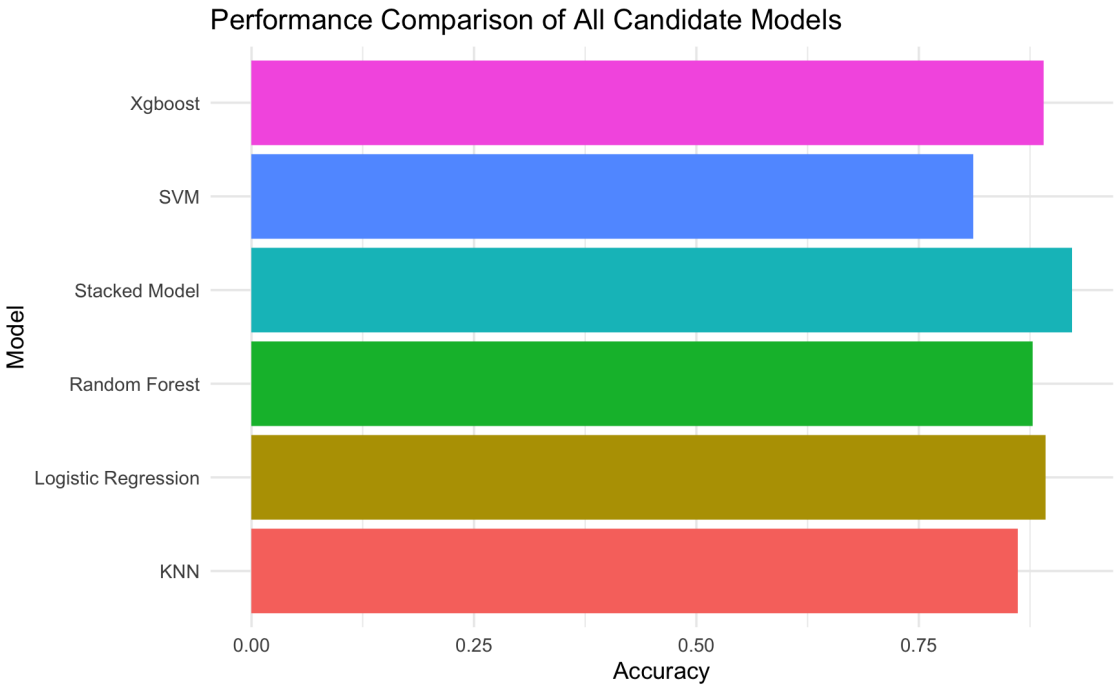

Figure 10: Model Comparison for classification task: Accuracy

From Figure 10, we can easily see that Random Forest Model perform much smaller RMSE than other models. Less complicated models including Ridge, Linear Regression and Lasso Regression, perform larger RMSE, which indicate a relatively bad performance.

We summarize the performance of each model to fit the training dataset in the following table, using Accuracy as the metric score.

5 Discussion of final model

The final model we decided to adopt effectively utilizes tidymodel stacking, sequentially stacking a boosted tree, a random forest, and a logistic regression. This ensembling method enables us to exploit the advantages of various models and combine them together, generating a powerful new model that outperforms traditional models in terms of accuracy, which is 0.92236. Compared with other five candidate models, our stacked model has the advantages of capturing a variety of data patterns and complex relationships and reducing overfitting. Also, stacking helps in balancing the

Table 2: Summary of Model Performance

Model Identifier	Metric Score (Accuracy)	SE of Metric
Model 1 (Logistic Regression)	0.89279	0.006407980
Model 2 (Random Forest)	0.878	0.006778850
Model 3 (XGBoost)	0.890	0.006480675
Model 4 (KNN)	0.861	0.007165357
Model 5 (SVM)	0.811	0.008109054
Model 6 (Stacking)	0.92236	0.005542706

bias-variance trade-off. Logistic Regression has high bias and low variance, while Random Forest has lower bias but higher variance, so our stacked model combining these can lead to better generalization on unseen data. Moreover, each submodel being stacked is capable of correcting the mistakes of each other, making our model promising when the dataset has a high level of complexity.

Despite the advantages of our stacked model, we realize that it has potential drawbacks simultaneously. First, as there are many parameters involved in this complex model, it is more time-consuming and complex to train and tune this model compared to non-stack single models. Second, although the model achieves good performance in the Kaggle competition, it is challenging to interpret feature importance due to the sophisticated nature of the stacked model.

Therefore, we shall try to improve our stacked model by more systematically tuning all important hyperparameters both manually and through functions like tune_grid(), as well as by stacking more kinds of classification models including neural network and KNN. We shall also explore if additional data such as the potential relationships among different columns in the training set will be useful. In this way, our stacked model will be further promoted and is promising to achieve a higher accuracy score.

6 Appendix 1: Annotated R Script

```
# Load needed libraries
library(tidymodels)
library(tidyverse)
library(xgboost)
library(stacks)

# Read training data, delete name col to make sure train and test are
   the same
train <- read_csv("train_class.csv")
train <- train %>% select(-name)
test <- read_csv("test_class.csv")

# do initial preprocessing, do mean imputation for all cols with NAs
col_imp <- c('income_per_cap_2016', 'income_per_cap_2017',
             'income_per_cap_2018', 'income_per_cap_2019',
```

```
             'income_per_cap_2020', 'gdp_2016', 'gdp_2017',
             'gdp_2018', 'gdp_2019', 'gdp_2020')
train <- train %>%
  mutate(across(col_imp, ~ ifelse(is.na(.), mean(., na.rm = TRUE), .)))
test <- test %>%
  mutate(across(col_imp, ~ ifelse(is.na(.), mean(., na.rm = TRUE), .)))
# convert winner into factors
train <- train %>%
  mutate(winner = as.factor(winner))

# Set seed, do 10 fold cross validation, where strata is winner to be
   predicted
set.seed(101)
folds <- vfold_cv(train, v = 10, strata = winner)

# create recipe after imputation, deal with zero variance,
   normalization, and dummys
recipe <- recipe(winner ~ ., data = train) %>%
  step_zv(all_predictors()) %>%
  step_dummy(all_nominal_predictors(), -all_outcomes()) %>%
  step_normalize(all_numeric_predictors())

# set the first model boost tree xgboost with all hyperparameters
   settled
bt_model <-
  boost_tree(
    mtry = 17,
    min_n = 50,
    trees = 230,
    tree_depth = 6,
    loss_reduction = 1.176322,
    learn_rate = 0.1,
    sample_size = 0.9968396,
    stop_iter = 5
  ) %>%
  set_mode("classification") %>%
  set_engine("xgboost")

# set the second model ramdom forest with all hyperparams tuned
rf_model <-  rand_forest(
  mtry = 11,
  min_n = 50,
  trees = 120,
) %>%
  set_mode("classification") %>%
  set_engine("ranger")

# set the third model, logistic regression
```

```
lg_model <- logistic_reg() %>%
  set_mode("classification") %>%
  set_engine("glm")

# define workflows for three models
bt_workflow <- workflow() %>% add_model(bt_model) %>% add_recipe(recipe
   )
rf_workflow <- workflow() %>% add_model(rf_model) %>% add_recipe(recipe
   )
lg_workflow <- workflow() %>% add_model(lg_model) %>% add_recipe(recipe
   )

# control setting for stack grid
ctrl_grid <- control_stack_grid()

# tune models using tune_grid
bt_res <- tune_grid(
  bt_workflow,
  resamples = folds,
  control = ctrl_grid
)

rf_res <- tune_grid(
  rf_workflow,
  resamples = folds,
  control = ctrl_grid
)

# fit logistic regression, no hyper params to tune here
lg_res <- fit_resamples(
  lg_workflow,
  resamples = folds,
  control = ctrl_grid
)

# create and initialize the stack, stack up all three models per
   documentation
mstack <- stacks() %>%
  add_candidates(bt_res) %>%
  add_candidates(rf_res) %>%
  add_candidates(lg_res) %>%
  blend_predictions() %>%
  fit_members()

# use the final model to do prediction, get result and final csv
pred <- test %>% select(id) %>%
  bind_cols(predict(mstack, new_data = test,
                    type = "class") %>% rename(winner = .pred_class))
```

```
write_csv(pred, "prediction_classify.csv")
```

7 Appendix 2: Contributions

Zixuan Liu: Created and tuned XGBoost model. Wrote part 4 candidate models and tuning in the report, comparing six models on their performance. Created tables to compare different models on their hyperparameters and accuracy.

Xinyang Wang: Carried out a thorough Exploratory Data Analysis and created a number of visualizations that showed the characteristics and distribution of the data set. Wrote the Introduction and entire Exploratory Data Analysis, including the illustrations and the detailed descriptions.

Yichen Wang: Conducted extensive exploration on classification models, implemented the final stacked model, tuned all needed parameters through functions and manual tuning. Wrote the Data Preprocesing, SVM and the stacked model part of the Model Tuning and Evaluation, Final Model Discussion, and Final R Script, all along with detailed interpretations.

Jessica Zhao: Explored model as candidate choices and created the logistic regression and KNN model and tuned them to perform well. Contributed to writing the model evaluation part. Wrote the table in the documentary form for comparison and evaluation.